

Jones Automation - Exercise - Answers

A: UI Mock-up Review & Identified Problems

Review of "Account Information" Widget:

- **Security & Compliance (Critical):**
 - **PCI-DSS Compliance Breach:** The form asks for Card Number, Expiration Month/Year, and Cardholder Name on a standard web form. For a global SaaS company, embedding these fields directly into a native form without using an iframe/tokenization service (like Stripe Elements or PayPal SDK) is a severe compliance violation. The backend should never 'touch' or look at raw credit card numbers.
 - **Missing CVV/CVC Field:** There is no field for the Card Verification Value (the 3-4 digit security code on the back of the card). Without this, transactions will be flagged as high-risk or outright rejected by payment gateways.
- **Usability & User Experience:**
 - **Card Number Constraints:** The label says "No dashes or spaces". This forces the user to accommodate the machine. A good UX should let users type or paste dashes/spaces and use frontend regex to strip them out automatically.
 - **Postal Code Limitations:** Label states "no dashes". Similar to the card number, this creates bad friction for international users (e.g., UK or Canadian postcodes that naturally contain letters/spaces, or US zip+4 with dashes).
 - **Middle Initial (MI) Field Placement:** The MI text box is awkwardly squished between First Name and Last Name. It breaks the grid alignment and can be confusing or ignored by non-US users.
 - **State/Province & Missing Country Field:** Despite being described as a global SaaS, the form entirely missing a "Country" field. This is a major architectural oversight. Without knowing the user's country, the payment gateway cannot accurately perform Address Verification System (AVS) checks, leading to high decline rates for international credit cards. Furthermore, its absence prevents the UI from dynamically adapting regional fields (e.g., hiding the "State" field for European users or changing "Postal Code" validation rules per region).

- **Unlabeled Secondary Address Field:** The input box beneath the "Credit Card Billing Street Address" lacks a label or placeholder. This causes UX confusion and violates basic web accessibility standards, as screen readers cannot identify the field's purpose.
- **Performance & Data Integrity:**
 - **Payment Amount as Static Text:** The Payment Amount is displayed as 30.00 but lacks a currency symbol (\$, €, ₪). A global company must explicitly state the currency.

B: Sample Test Cases

Here are 3 functional test cases designed for this billing functionality:

Test Case 1: Valid Credit Card Submission (Happy Flow)

- **Objective:** Verify that a user can successfully complete a transaction using valid details.
- **Preconditions:** User is on the billing widget page, payment amount is loaded (\$30.00).
- **Steps:**
 1. Select "VISA" from Card Type.
 2. Enter a valid, simulated 16-digit Visa card number.
 3. Select valid future Expiration Month and Year.
 4. Enter "Amy" in First Name and "Winehouse" in Last Name.
 5. Enter valid billing street address, city, state, and postal code.
 6. Click the "Continue" button.
- **Expected Result:** Form submits successfully, API responds with a successful authorization token, and user is redirected to a "Payment Successful" confirmation page.

Test Case 2: Validation of Past Expiration Date (Negative Flow)

- **Objective:** Verify that the system prevents submission of an expired credit card.
- **Steps:**
 1. Fill all required fields with valid information.
 2. Under "Expiration", select a past month/year (e.g., May 2024).
 3. Click "Continue".
- **Expected Result:** Form submission is blocked. An inline error message appears under the Expiration fields stating: *"The expiration date must be in the future."* No API call is fired to the gateway.

Test Case 3: International Character Validation in Names & Addresses (Edge Case)

- **Objective:** Ensure the system supports international global users utilizing special characters.
- **Steps:**
 1. Enter a valid card number and future expiration date.
 2. In "First Name" and "Last Name", enter names with accents/diacritics (e.g., *François* or *Müller*).
 3. In "Credit Card Billing Street Address", enter international characters (e.g., *Champs-Élysées*).
 4. Click "Continue".
- **Expected Result:** The system accepts the special characters without throwing formatting errors, sanitizes the inputs correctly, and successfully passes the data to the payment processor.

C: Product Solution for the Most Severe Bug

The Most Severe Bug: PCI-DSS Security Compliance & Missing CVV/CVC Field.

Suggested Product Solution: Instead of rendering local HTML `<input>` fields for capturing highly sensitive cardholder data, the company should integrate a third-party hosted payment fields solution (e.g., **Stripe Elements** or **Braintree Hosted Fields**).

How it works conceptually:

1. The Card Number and Expiration fields will be replaced by secure iframes hosted directly on the payment provider's servers.
2. Visually, it will look like a seamless part of the SaaS application layout, keeping the UI clean.
3. The payment gateway provider will automatically add the missing **CVV/CVC** field into their secure iframe layout.
4. Upon clicking "Continue", the script intercepts the submit action, securely exchanges the raw card data from the iframe for a safe **Token** (tokenization) and passes only that token to our backend server.

Business Impact: This completely removes the SaaS company from the PCI scope, protects the company from data leaks, and boosts user trust during checkout.